

Experiment 2: Behavioral model of 1-bit comparator.

VHDL Program:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

ENTITY exp2_391 is
    Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          C : inout STD_LOGIC;
          D : out  STD_LOGIC;
          E : inout STD_LOGIC);
end exp2_391;

architecture Behavioral of exp2_391 is

begin
    C <= NOT(A) AND B;
    E <= A AND NOT(B);
    D <= C NOR E;

end Behavioral;
```

Experiment 3: VHDL Programs for half adder and full adder.

Half Adder:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

ENTITY halfAdder_391 is
    Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          S : out  STD_LOGIC;
          C : out  STD_LOGIC);
end halfAdder_391;

architecture Behavioral of halfAdder_391 is

begin
    Process(A, B)
    begin
        if(A='0' AND B='0') then
            S <= '0';
            C <= '0';
        elsif(A='0' AND B='1') then
            S <= '1';
            C <= '0';
        elsif(A='1' AND B='0') then
            S <= '1';
            C <= '0';
        elsif(A='1' AND B='1') then
            S <= '0';
            C <= '1';
        end if;
    end Process;

end Behavioral;
```

Full Adder:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

ENTITY fullAdder_391 is
    Port ( A : in  STD_LOGIC;
          B : in  STD_LOGIC;
          Cin : in  STD_LOGIC;
          S : out  STD_LOGIC;
          Cout : out  STD_LOGIC);
end fullAdder_391;

architecture Behavioral of fullAdder_391 is

    SIGNAL X,Y,Z: STD_LOGIC;

begin

    X <= A XOR B;
    Y <= X AND Cin;
    Z <= A AND B;
    S <= X XOR Cin;
    Cout <= X OR Z;

end Behavioral;
```

Experiment 4: VHDL program for 4:1 Multiplexer and 1:4 Demultiplexer.

4:1 Multiplexer:

```
library ieee;
use ieee.std_logic_1164.ALL;

ENTITY Mux is
    port( I3 : in std_logic_vector(2 downto 0);
          I2 : in std_logic_vector(2 downto 0);
          I1 : in std_logic_vector(2 downto 0);
          I0 : in std_logic_vector(2 downto 0);
          S : in std_logic_vector(1 downto 0);
          O : out std_logic_vector(2 downto 0));
end Mux;

architecture Behavioral of Mux is

begin

    process(I3,I2,I1,I0,S)
    begin
        case S is
            when "00" => O <= I0;
            when "01" => O <= I1;
            when "10" => O <= I2;
            when "11" => O <= I3;
            when others => O <= "ZZZ";
        end case;
    end process;
end Behavioral;
```

1:4 Demultiplexer:

```

library ieee;
use ieee.std_logic_1164.ALL;

ENTITY DECODER is
    port( I : in std_logic_vector(1 downto 0);
          O : out std_logic_vector(3 downto 0));
end DECODER;

architecture behv of DECODER is

begin

    process (I)
    begin
        case I is
            when "00" => O <= "0001";
            when "01" => O <= "0010";
            when "10" => O <= "0100";
            when "11" => O <= "1000";
            when others => O <= "XXXX";
        end case;
    end process;
end behv;

architecture when_else of DECODER is

begin
O <= "0001" when I = "00" else
    "0010" when I = "01" else
    "0100" when I = "10" else
    "1000" when I = "11" else
    "XXXX";
end when_else;

```

Experiment 5: Write program for behaviour model of BDC-to-seven segment display.

VHDL Program:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

ENTITY bcd_7segment is
    Port ( BCDin : in STD_LOGIC_VECTOR (3 downto 0);
          Seven_Segment : out STD_LOGIC_VECTOR (6 downto 0));
end bcd_7segment;

architecture Behavioral of bcd_7segment is

begin

    process(BCDin)
    begin
        case BCDin is
            when "0000" =>
                Seven_Segment <= "1111110";
            when "0001" =>
                Seven_Segment <= "0110000";
            when "0010" =>
                Seven_Segment <= "1101101";

```

```

        when "0011" =>
            Seven_Segment <= "1111001";
        when "0100" =>
            Seven_Segment <= "0110011";
        when "0101" =>
            Seven_Segment <= "1011011";
        when "0110" =>
            Seven_Segment <= "1011110";
        when "0111" =>
            Seven_Segment <= "1110000";
        when "1000" =>
            Seven_Segment <= "1111111";
        when "1001" =>
            Seven_Segment <= "1111011";
        when others =>
            Seven_Segment <= "1111111";
    end case;
end process;

end Behavioral;

```

Experiment 6: Write VHDL program for behaviour model of 3-bit counter.

VHDL Program:

```

library ieee;
use ieee.std_logic_1164.ALL;
use ieee.std_logic_unsigned.ALL;

ENTITY counter is
    port(clock : in std_logic;
          clear : in std_logic;
          count : in std_logic;
          Q : out std_logic_vector(2 downto 0));
end counter;

architecture behv of counter is

    signal Pre_Q: std_logic_vector(2 downto 0);

begin

    process(clock, count, clear)
    begin
        if clear = '1' then
            Pre_Q <= Pre_Q - Pre_Q;
        elsif (clock='1' and clock'event) then
            if count = '1' then
                Pre_Q <= Pre_Q + 1;
            end if;
        end if;
    end process;

    Q <= Pre_Q;
end behv;

```

Experiment 7: Write a VHDL program for behaviour model 1101 sequence detector.

VHDL Program:

```
library ieee;
use ieee.std_logic_1164.ALL;

ENTITY seq_design is
    port( din: in std_logic;
          clk: in std_logic;
          clr: in std_logic;
          dout: out std_logic);
end seq_design;

architecture FSM of seq_design is

    type state_type is (S0, S1, S2, S3, S4);
    signal next_state, current_state: state_type;

begin

    state_reg: process(clk, clr)
    begin
        if (clr='1') then
            current_state <= S0;
        elsif (clk'event and clk='1') then
            current_state <= next_state;
        end if;
    end process;

    comb_logic: process(current_state, din)
    begin
        case current_state is
            when S0 =>
                if din='0' then
                    next_state <= S0;
                elsif din='1' then
                    next_state <= S1;
                end if;
            when S1 =>
                if din='0' then
                    next_state <= S0;
                elsif din='1' then
                    next_state <= S2;
                end if;
            when S2 =>
                if din='0' then
                    next_state <= S3;
                elsif din='1' then
                    next_state <= S2;
                end if;
            when S3 =>
                if din='0' then
                    next_state <= S0;
                elsif din='1' then
                    next_state <= S4;
                end if;
            when S4 =>
                if din='0' then
                    next_state <= S0;
                elsif din='1' then
```

```
        next_state <= S2;
    end if;
    when others =>
        null;
    end case;
end process;

comb_logic_2 : process(current_state)
begin
    if current_state = S4 then
        dout <= '1';
    else
        dout <= '0';
    endif;
end process;

end FSM;
```